

Blueprint Técnico Opinonado: Copiloto de Trading de Cripto en 2026

Tesis del producto. El copiloto no predice precios. Hace tres cosas, todas verificables: (1) **estructura el criterio del trader** vía conversación con contexto de mercado en tiempo real; (2) **ejecuta análisis cuantificables on-demand** (indicadores, confluencias multi-timeframe, derivados, on-chain) con citación de las herramientas que usó; (3) **gestiona research reproducible** (backtests con walk-forward y CPCV, journal automático, detección de sesgos). El LLM es el intérprete y el orquestador, no el oráculo. Cualquier diseño que confunda esto fracasa por construcción.

A continuación, el blueprint completo, accionable la próxima semana.

1. Stack de LLM y agente

1.1 Modelo “cerebro” principal: Claude Sonnet 4.6 como caballo de batalla, Claude Opus 4.7 como fallback para análisis profundo

Recomendación dura. El backbone es **Claude Sonnet 4.6** vía la Anthropic API (también disponible en Bedrock y Vertex). Sonnet 4.6 entrega ~98 % de la calidad de Opus a una fracción del coste y latencia, con tool-use nativo de altísima fiabilidad. Para queries explícitamente marcados como “deep dive” (research de estrategias, redacción de tesis macro, post-mortem de pérdidas grandes) escalas a **Claude Opus 4.7** (\$5/\$25 por millón input/output, contexto ~1M). [ChatGPT AI Hub](#)

Por qué no GPT-5.5. Es brillante en razonamiento puro y Terminal-Bench, pero (a) el doblaje de precio a \$5/\$30 lo hace antieconómico para un agente conversacional con tool-loops largos, (b) en los benchmarks MCP-Atlas (orquestación de tools encadenados) Claude lidera 79.1 % vs 75.3 %, [Build Fast with AI](#) y eso es exactamente lo que necesitas, (c) en HLE sin tools Opus 4.7 lidera 46.9 % vs 41.4 %, [Build Fast with AI](#) y los problemas de trading son razonamiento + recall, no AIME.

Por qué no Gemini 3.1 Pro. Tiene el contexto más largo (2M) y es el más barato (\$2/\$12), pero su lead es en multimodal/vídeo, que no usas, y queda último en MCP-Atlas (73.9 %). Lo dejas como **fallback de coste** para tareas batch (resúmenes nocturnos, embeddings de news a escala).

Por qué no open-source (Llama 3.3, Qwen3 Max, DeepSeek V4). Para uso personal con derivados reales en juego, el coste marginal de la API frontier es ridículo comparado al riesgo operativo de un modelo peor en tool calling. DeepSeek V4 / Qwen3 Max son competitivos en razonamiento, pero la fiabilidad del JSON estructurado y la latencia P95 de la Anthropic API en producción son materialmente mejores. Reserva self-hosting para cuando escales a usuarios externos y la economía cambie.

Routing concreto (un AI Gateway delante).

Tarea	Modelo
Chat conversacional con tool-loops	claude-sonnet-4-6
Análisis técnico on-demand (multi-tool)	claude-sonnet-4-6
Research de estrategias / journal post-mortem	claude-opus-4-7
Resumen nocturno de news / embeddings batch	gemini-3.1-pro
Razonamiento cuantitativo aislado (math heavy)	gpt-5.5 con effort=high
Embeddings	voyage-3-large (ver §2)

Implementa el gateway con **OpenRouter** o, mejor para control, con **LiteLLM** self-hosted detrás de tu FastAPI. Logea cost/latency/error por request a Postgres y rota a fallback en error.

1.2 Framework de agente: Pydantic AI, con escalada a LangGraph sólo cuando el grafo lo justifique

Recomendación dura. Empieza con **Pydantic AI** (≈16k stars, modelo-agnóstico, type-safe nativo, integración FastAPI directa). Para un copiloto donde cada respuesta del modelo termina siendo validada como una `Pydantic BaseModel` (señal con score, plan de trade, parámetros de backtest), es la elección correcta. La validación estructurada no es un nice-to-have: es la diferencia entre que el LLM diga “compra BTC” y que entregue `TradeIdea(side='long', entry=..., invalidation=..., r_multiple=..., confidence=0.62, reasons=[...])` validado o re-prompteado automáticamente.

Cuándo migras componentes a LangGraph. Sólo cuando un workflow se convierta en una *máquina de estados con ciclos y human-in-the-loop* — típicamente, el módulo de research de estrategias (idear → backtest → walk-forward → CPCV → review humano → live paper). Ahí el grafo explícito + checkpointing de LangGraph se gana su sitio. Para todo lo demás (chat, análisis de un activo, scan), Pydantic AI es más limpio.

Lo que NO recomiendo.

- **CrewAI:** paradigma “roles que conversan” suena bien en demos, pero quema tokens y oculta dónde fallan las cosas. Anti-patrón para trading.
- **AutoGen:** en mantenimiento (absorbido por Microsoft Agent Framework), seguridad casi nula. No.
- **Custom con tool-use crudo de Anthropic API:** lo descartas porque pierdes (a) provider-agnostic, (b) validación tipada, (c) tracing barato. El SDK de Pydantic AI ya envuelve la API nativa.

1.3 Sistema de tools: contrato mínimo viable

Diseña tools como **funciones puras, idempotentes, con outputs Pydantic**. Cada tool retorna `data + provenance` (timestamp, fuente, URL/exchange). Esto habilita citación y auditoría.

Catálogo opinado de tools (v1):

```
# Datos de mercado
get_ohlcv(symbol, timeframe, limit, exchange='binance')
get_orderbook(symbol, depth=50, exchange)
get_recent_trades(symbol, limit, exchange)
get_ticker(symbol, exchange)

# Indicadores (calculados sobre OHLCV crudo, no proxies a TV)
get_indicators(symbol, timeframe, indicators=['ema20','ema50','rsi','atr','macd',
get_market_structure(symbol, timeframe) # pivots, S/R, swing highs/lows
get_multi_tf_confluence(symbol, timeframes=['15m','1h','4h','1d'])

# Derivados (críticos en cripto)
get_funding_rate(symbol, exchange='all') # vía Coinglass + ccxt directo
get_open_interest(symbol, window='24h')
get_liquidations(symbol, window='1h')
get_long_short_ratio(symbol, exchange)
get_basis(symbol) # perp vs spot, futuros calendar

# On-chain
get_onchain_metric(asset, metric, window) # SOPR, MVRV, exchange netflow
get_whale_movements(asset, threshold_usd, window)

# News / sentiment
get_news(symbol_or_topic, since, sources=['cryptopanic','tradfi'])
get_social_sentiment(symbol, window)
```

```

# Backtesting / research
run_backtest(strategy_id, symbol, timeframe, since, until, fees, slippage_bps)
run_walk_forward(strategy_id, n_splits, ...)
run_cpcv(strategy_id, n_groups, k_test, embargo_pct)
get_strategy_metrics(backtest_id) # Sharpe, deflated Sharpe, MAR, Ulcer, PSR

# Scanner / alertas
scan_market(filter_dsl) # "RSI<30 AND vol_z>2 AND funding<0"
create_alert(rule_dsl, channels=['ui', 'telegram'])

# Memoria / journal
log_trade(trade_data)
get_trader_profile() # estilo, risk tolerance, instrumentos preferidos
get_recent_trades(window)
detect_bias_patterns(window)

```

Patrones obligatorios para tools:

- Cada tool tiene **timeout** (3 s para REST, 200 ms para cache de redis).
- Cada tool retorna `ToolResult[T]` con `data: T`, `source: str`, `as_of: datetime`, `warnings: list[str]`.
- Tools de datos de mercado **siempre** consultan cache Redis primero (TTL = timeframe/4 con un piso de 5 s para 1m bars).
- **Nunca** un tool ejecuta órdenes reales en v1. El copiloto es asesor, no broker. La acción "place_order" se introduce después de paper-trading verificado y con un confirmation handshake.

1.4 Memoria

Tres capas, claras y separadas:

1. **Working memory (turno conversacional):** los últimos N mensajes en context window. Trivial.
2. **Session memory:** lo que el trader ha mirado hoy (símbolos, timeframes, hipótesis abiertas). Redis con TTL 24h. Inyectada como system prompt al inicio de cada request.
3. **Long-term trader profile:** Postgres + opcionalmente RAG. Almacena (a) preferencias declaradas (timeframes, max leverage tolerado, mercados), (b) estilo inferido (avg holding time, win rate por setup, sesgos detectados), (c) journal embebido para retrieval semántico ("¿qué me pasó la última vez que abrí long en ETH con funding negativo extremo?"). Esto es lo que diferencia un copiloto real de un wrapper de ChatGPT.

Para conversación persistente tipo "Claude Code": mantén un `MEMORY.md` por trader que el agente puede leer/escribir mediante tools `read_memory()` / `update_memory(section, content)` . Patrón de Pydantic Deep Agents y muy efectivo en la práctica.

2. RAG para trading

2.1 ¿Vale la pena? Sí, pero acotado.

RAG general-purpose ("índexame todo Murphy + Elder + papers") es un **anti-patrón** para trading: degrada la respuesta porque inyecta contexto educacional cuando el trader necesita criterio actual sobre BTC ahora. RAG sí gana fuerte en tres casos concretos:

1. **Journal personal del trader** (alto valor; tu corpus único). Cada trade cerrado se embebe con `(setup, outcome, mistakes, market_regime)` . Es tu mejor fuente de auto-conocimiento.
2. **Playbook de estrategias propias** (reglas formales de cada setup que tú mismo defines). Sirve para que el agente invoque la regla correcta cuando reconoce un patrón.
3. **Glosario / docs internas** (definiciones de indicadores no estándar, parámetros calibrados, mappings de símbolos). Reduce alucinaciones de notación.

Lo que NO indexes en v1:

- Murphy, Elder, libros TA. El conocimiento ya está en el LLM frontier.
- Papers académicos completos (López de Prado, Bailey). En su lugar, escribe tu propio "memo" de takeaways y eso lo indexes.
- Transcripts de macro analysts y X/Twitter. Demasiado ruido y obsolescencia rápida; usa news tools en tiempo real.

2.2 Vector DB: pgvector (con pgvector scale de Timescale si crece)

Recomendación dura. Ya vas a tener Postgres + TimescaleDB para series temporales (§6). Añadir `pgvector 0.9` + la extensión `pgvector scale` te da búsqueda vectorial con DiskANN + binary quantization que en benchmarks 2026 rinde 471 QPS @ 99 % recall sobre 50M vectores — competitivo con Pinecone y dominante sobre Qdrant a esa escala (firecrawl benchmark, Timescale). Para tu corpus realista (<5M vectores), es masivamente suficiente.

Por qué no Qdrant / Pinecone / Weaviate / LanceDB.

- **Qdrant:** excelente, pero añade un servicio. Sólo tiene sentido si necesitas filtrado complejo + late-interaction (ColBERT) — no es tu caso aún.

- **Pinecone:** managed perfecto, pero \$70-300/mes `4xxi` para nada que no resuelva pgvector. Desperdicio.
- **Weaviate:** GraphQL + módulos pesan; Java runtime.
- **LanceDB:** prometedor para datasets grandes embebidos, pero ecosistema más joven y no aporta sobre pgvector a tu escala.
- **Chroma:** dev-tool. Para producción personal vale, pero ya que tienes Postgres, no añadas otra cosa.

2.3 Chunking + embeddings

Embedding model: `voyage-3-large` (Voyage AI, 65.1 MTEB, \$0.12/M tokens). En el dominio financiero/legal/code hay un gap medible de 4-6 puntos NDCG vs OpenAI text-embedding-3-large y Cohere embed-v4. `TokenMix` El journal del trader es texto especializado — cuando preguntes “trades similares al actual setup”, quieres precisión. Voyage también ofrece **shared embedding space** entre modelos (puedes indexar con `large` y consultar con `lite` sin re-indexar). `BuildMVPFast` Si el coste preocupa, `voyage-3-lite` sigue siendo competitivo.

Alternativa razonable: `text-embedding-3-large` de OpenAI (\$0.13/M, 3072 dim). Diferencia sólo se nota en queries específicas; si ya pagas OpenAI por algo, mantenlo simple.

No uses `text-embedding-3-small` para journal: la diferencia de calidad es real cuando el corpus es pequeño y los matices importan.

Chunking: trades del journal → 1 trade = 1 chunk con metadata estructurada (`symbol`, `side`, `entry`, `exit`, `pnl_r`, `setup_tag`, `regime_tag`, `mistakes`). Documentos largos (memos macro) → semantic chunking de ~500 tokens con overlap de 80, **no** chunking ciego por caracteres.

2.4 Anti-alucinación

Tres mecanismos no-negociables:

1. **Hybrid search obligatoria.** BM25 (Postgres `tsvector` o `paradedb`) + dense (pgvector), fusionados con Reciprocal Rank Fusion. Sin esto, queries con tickers/símbolos exactos fallan (los embeddings no clusterizan bien strings tipo `BTC-PERP-USDT`).
2. **Reranking con** `Cohere rerank-3` o `Voyage rerank-2` . Recuperas top-50, reranqueas, devuelves top-5 al LLM. Reduce hallucinations medidas en ~35 % (Databricks 2025).

`ZeroEntropy`

3. **Citation forcing en el prompt.** El system prompt obliga: "Cada afirmación cuantitativa va acompañada de `[tool:get_indicators]` o `[journal:trade_id=...]` . Si no tienes fuente, dilo explícitamente." Combina con un validator Pydantic que rechaza outputs sin citas para campos críticos (precio, niveles, métricas).
-

3. Datos de mercado e indicadores

3.1 TradingView (MCP) vs cálculo propio: **cálculo propio, sin dudas**

Recomendación dura. Calcula todos los indicadores tú mismo sobre OHLCV crudo de los exchanges. **No** dependas de un MCP a TradingView para análisis programático.

Razones:

- **Reproducibilidad y backtesting:** necesitas que la fórmula sea idéntica entre tiempo real y backtest. Si TV cambia un default de período, tu backtest miente.
- **Licencias:** TradingView Charting/Trading Library no se licencia para uso privado/personal — sólo a empresas con producto público [TradingView](#) (cláusula explícita en el license agreement). Para el render de gráficos sí puedes usar **Lightweight Charts** (Apache 2.0, libre para uso personal). Para cálculo de indicadores, ni intentes scrapear; viola TOS y es frágil.
- **Latencia y coste:** REST a TV vs cálculo local sobre Polars es 10–100x más lento y depende de su uptime.
- **Control:** cuando el LLM diga "RSI(14) está en 28", quieres saber **exactamente** qué fórmula. TradingView usa SMMA para RSI; otros usan EMA. Tú decides.

3.2 Librería de indicadores: `pandas-ta-classic` + `TA-Lib` **cuando exista, todo orquestado con Polars**

El estado a abril 2026 es desordenado:

- **TA-Lib:** C, máximo rendimiento, [Sling Academy](#) cobertura extensa de candlestick patterns, pero binding C requiere compilar; estable pero con poca evolución reciente.
- `pandas-ta` (twopirllc original): **mantenimiento inactivo** según Snyk, [Snyk](#) releases anuales con bugs acumulados.
- `pandas-ta-classic` (fork por [xgboosted](#) , PyPI release marzo 2026): es el **fork vivo** que debes usar.
- `tulipy` : limitada y semi-abandonada. Skip.

- `finta` : educativa, sin coverage. Skip.
- `polars-ta` : prometedor por velocidad (Polars + Rust), cobertura aún parcial, pero mejor opción si tu DataFrame engine es Polars.

Stack final:

- Backend de datos en **Polars LazyFrames** (no pandas; el GIL y la velocidad importan cuando escaneas 200 símbolos simultáneamente).
- `polars-ta` para indicadores comunes (EMA, RSI, ATR, MACD, BB, ADX, VWAP).
- `TA-Lib` (binding Python) para candlestick patterns y los pocos exóticos no en `polars-ta`.
- `pandas-ta-classic` como red de seguridad para indicadores raros.
- Implementaciones **propias** y testeadas para: pivots clásicos/Fibonacci, order blocks (definición operacional clara, no la mística de YouTube), liquidity zones (basadas en agregación de volumen y wick rejections), Volume Profile (VPVR/VWAP por sesión).

3.3 Exchanges/APIs: CCXT Pro + adapters nativos para Binance y Bybit

Recomendación. CCXT Pro (~\$300/año) es la opción correcta como capa unificada para 70+ exchanges con WebSocket nativo. Pero para los dos exchanges donde tendrás más volumen (Binance Futures y Bybit) **además** mantén un thin adapter directo a su WS oficial. Razón: cuando un mercado se mueve fuerte, los reconnects y el delta-decoding del orderbook hechos a medida son más fiables que la abstracción genérica.

Reglas:

- **WebSockets para todo lo "live"**: trades, orderbook, klines en curso, funding, liquidations, marks. REST sólo para histórico y backfill.
- **Snapshot + diff** para orderbook (estándar Binance/Bybit). No reconstruyas con `fetch_order_book` cada N segundos — eso es trader nivel hobby.
- **OHLCV histórico**: usa REST con paginación; cachea en TimescaleDB con `INSERT ... ON CONFLICT DO NOTHING`.
- Optimizaciones de CCXT que sí marcan diferencia: instala `orjson` y `coincurve` (firma ECDSA pasa de 45 ms a 0.05 ms [GitHub](#) [npm](#) — relevante en order placement futuro).

3.4 Indicadores: señal vs ruido en crypto 2026

Mi opinión, calibrada por lo que sigue funcionando:

Mantener (señal):

- **Tendencia:** EMA 21/55/200, ADX (>20 valida tendencia). MACD para confirmar momentum cruces.
- **Momentum:** RSI con divergencias multi-TF — no el level “80=overbought” naïf, sino divergencia + estructura.
- **Volatilidad: ATR es el rey.** Define stops y position sizing en $R = ATR \times k$. Bollinger Bands como contexto de squeeze.
- **Volumen: VWAP intradía** (institutional anchor). Volume Profile (POC, value area). OBV es subóptimo en crypto 24/7 — ignóralo.
- **Estructura:** swings (HH/HL/LH/LL), niveles horizontales por confluencia de wicks, S/R psicológico.

Ruido / sobrevalorado:

- Estocástico, CCI, Williams %R: redundantes con RSI.
- Ichimoku: estéticamente atractivo, marginalmente útil en crypto, sobre-fit del backtest japonés.
- “Order blocks” / “FVG” / “liquidity sweeps” tal como los vende TikTok: **operacionalízalos rigurosamente** con reglas de detección (alta volatilidad + impulsivo + retest), o no los uses. El LLM puede ayudar a detectarlos si le das la regla, no la mística.
- Patrones armónicos (Gartley, etc.): casi todos sobrevivientes de survivorship bias.

3.5 Datos de derivados: críticos, no opcionales

Para crypto, los derivados son **leading indicators del flujo retail/whale**. Stack:

- **Funding rates** (perpetuals): vía CCXT directo (`fetch_funding_rate`) y **Coinglass API** (\$35/mes plan básico → \$79+ para histórico OHLC) para datos cross-exchange agregados. Funding extremo (>0.05 % 8h o <-0.03 %) = setup de mean reversion potente combinado con price action.
- **Open Interest:** Coinglass para histórico OHLC, CoinGlass exchange APIs directas para snapshot. Divergencia OI/precio es señal real (rally con OI bajando = corto squeeze terminándose).
- **Liquidations:** Coinglass `liquidation heatmaps` y `liquidation maps` CoinGlass — son la señal que más alpha generó en 2024-2025 para timing de pivots de corto plazo. Inclúyelo.

- **Long/Short ratio** (top traders): proxy decente; úsalo de Binance/Bybit directo.
- **Basis** (perp vs spot, futures calendar): cálculo trivial; señal limpia de stress de mercado.

3.6 On-chain: alpha real pero acotado a horizontes medios/largos

Verdad incómoda: para day-trading puro, on-chain rara vez aporta. Para swing/posición sí.

- **Glassnode** (Advanced \$39/mes anual, Professional \$799/mes): el estándar para SOPR, MVRV, NUPL, exchange netflow. Worth it para BTC/ETH macro.
- **CryptoQuant:** complementario, especialmente Exchange Whale Ratio y miner outflows.
- **Nansen:** smart-money wallets. Útil para narrativas en altcoins; caro.
- **Santiment:** sentiment + dev activity. Marginal para trading puro.
- **Dune:** SQL sobre on-chain. Imprescindible para queries custom (dashboards específicos por protocolo).

Mi stack v1: Glassnode Advanced + CryptoQuant gratis/básico + Dune para queries ad-hoc. Salta Nansen hasta que estés operando alts donde wallet labels muevan la aguja.

3.7 News / sentiment

Cryptopanic API + un agregador de TradFi (Benzinga o Polygon News API) cubren 95 %.

Twitter/X via API es caro y noisy: \$100/mes mínimo y la calidad de feed bajó tras 2023.

Skip a menos que tengas un motivo concreto.

Importante: la news data **no se la das al LLM como dump**. La pasas por (a) clasificador de relevancia (puede ser una llamada a Sonnet con prompt corto), (b) deduplicación, (c) ranking por impacto. Lo que llega al chat es señal, no noise.

4. Backtesting y research

4.1 Framework: **VectorBT PRO** para investigación + **NautilusTrader** para producción/paper

Recomendación dura, doble stack. Son dos problemas distintos.

- **Investigación / sweep de parámetros / análisis robustez → VectorBT PRO** (commercial, ~€500/año pero vale cada euro). Numba/Rust + vectorización masiva, **VectorBT** 1000 backtests en el tiempo que otros hacen 1. **Greyhoundanalytics** Plotly + Jupyter widgets para análisis interactivo. **VectorBT** Es el único framework Python

donde puedes hacer combinatorial purged cross-validation sobre 500 combinaciones en minutos.

- **Validación realista pre-live + paper trading + futuro live → NautilusTrader** (Apache 2.0, core en Rust). Event-driven con paridad backtest/live (mismo código corre en ambos), `Python` modelado de orderbook L2/L3, latencias y fills realistas. Es la elección correcta para cualquier estrategia que vayas a operar.

Workflow recomendado: ideación e idea-screening en VectorBT PRO; cuando una estrategia pasa filtros, la **reimplementas** en NautilusTrader y verificas que las métricas no colapsan al añadir microestructura realista. Si colapsan, no era estrategia, era curva ajustada al simulador vectorizado.

Por qué no los demás:

- **Backtrader:** maduro pero **desarrollo activo paró ~2018**. `Greyhoundanalytics` Está en modo museo. Skip.
- `backtesting.py` : didáctico, OK para prototipos, no para algo serio.
- **Zipline-reloaded:** heredado de Quantopian, ergonomía rara, comunidad pequeña, mal fit para crypto 24/7.
- `vectorbt` **libre:** sólo mantenimiento; el desarrollo activo está en PRO. `Greyhoundanalytics`
- **Lean/QuantConnect:** enorme y serio, pero te ata a su cloud o C# embed; demasiado para uso personal.

4.2 Event-driven vs vectorizado

Regla simple:

- **Vectorizado** (VectorBT) cuando exploras: ¿este edge existe? ¿en qué timeframe? ¿con qué parámetros? Ignora microestructura, optimiza throughput.
- **Event-driven** (Nautilus) cuando validas: ¿sobrevive a slippage real, partial fills, queueing y latencia? Es lo que decide si vas live.

Ningún backtest vectorizado debe ser tu última palabra antes de paper-trade.

4.3 Walk-forward bien hecho

Implementación correcta:

1. Define ventana **in-sample** (e.g. 12 meses) y **out-of-sample** (3 meses) con **embargo** entre ellas ($\geq 1\%$ del dataset, o el largo del label si labels son forward-looking).

2. Optimiza parámetros sólo en in-sample.
3. Aplica con parámetros congelados en out-of-sample.
4. Roll forward: avanza ambas ventanas, repite. Genera **k segmentos OOS concatenados** que forman tu equity curve "real".
5. **Nunca** miras el OOS hasta el final. Si lo miras y reoptimizas, has hecho data snooping.

4.4 Métricas que importan vs vanity

Importan	Vanity / engañosas
Deflated Sharpe Ratio (Bailey & López de Prado 2014, ajusta por nº trials y no-normalidad) SSRN	Sharpe puro
Probabilistic Sharpe Ratio (PSR)	Win rate aislado
MAR / Calmar (CAGR / max DD)	Total return
Sortino (downside vol)	Avg trade
Ulcer Index	"Profit factor" sin contexto
Expectancy en R (avg R por trade)	Nº trades alto como prueba de robustez
Max DD + duración del DD	Sharpe anualizado de 3 meses de data
Distribución de trades (skew, kurt, tail)	Cherry-picked best-month

Regla: si una estrategia tiene Sharpe 2.5 pero deflated Sharpe 0.4, **no existe**. Está sobreajustada al universo de pruebas.

4.5 Anti-overfitting

Pipeline obligatorio antes de paper:

1. **Out-of-sample estricto** (walk-forward, ya descrito).
2. **Combinatorial Purged Cross-Validation (CPCV)** (López de Prado, *Advances in Financial Machine Learning* cap. 12). Implementación en `mlfinlab` o `timeseriescv` (Sam31415). Towards AI Genera **N paths** de equity en lugar de 1 — distribución de

Sharpe, no punto. [Wikipedia](#) Comparado con walk-forward, CPCV reduce sustancialmente la *Probability of Backtest Overfitting (PBO)* [Scribd](#) (Bailey et al. 2016).

3. **Monte Carlo** sobre orden de trades (bootstrap) para ver distribución de drawdowns.
4. **Sensitivity analysis**: $\pm 20\%$ en parámetros clave; si las métricas colapsan, hay edge pequeño y curve fit grande.
5. **Deflated Sharpe** explícitamente reportado con `N_trials` que tu sweep generó.
6. **PBO** computado.

Si esto suena overkill para "uso personal": no lo es. Es la diferencia entre operar un sistema y operar ruido reciclado.

4.6 Modelado de fricciones realistas

- **Fees**: usa fees reales de tu tier (taker en Binance USDT-M $\sim 0.04\%$, maker -0.005% rebates). En NautilusTrader configura `FillModel` con la fee estructura del exchange.
- **Slippage**: modelo $\%$ del `ATR(1m)` para market orders, e.g. $0.05 \times \text{ATR}$. Para limits, modela probabilidad de fill como función de distancia al best bid/ask y volumen.
- **Market impact**: para tamaño $< 0.5\%$ del volumen 1m de tu venue, ignorable. Por encima, usa modelo lineal (Almgren-Chriss simplificado).
- **Funding costs**: en perps, **suma/resta el funding** cada 8 h en backtest. Es la diferencia entre una estrategia long-bias rentable y una que regala todo a través del funding.
- **Borrow costs** (margin spot): si usas margin, modélalo.
- **Latency**: en Nautilus configura latencia simulada (50–150 ms típico desde un VPS bien ubicado).

5. Gráficas y frontend

5.1 Charts: TradingView Lightweight Charts, sin dudarlo

Recomendación. Lightweight Charts (Apache 2.0, ~ 45 KB, [TradingView](#) nativo HTML5 Canvas) es la respuesta. Es libre para uso personal, performante con miles de velas, [TradingView](#) y soporta el plugin system para dibujar overlays custom [GitHub](#) (señales del bot, drawings, áreas de S/R, marcadores de trades del journal). Mantenido directamente por TradingView. [TradingView](#)

Por qué no los demás:

- **TradingView Charting Library / Trading Platform Library:** licencia comercial, **explícitamente prohibida para uso personal.** TradingView No.
- **Plotly / ECharts / Highcharts / ApexCharts:** charts genéricos. Trabajo de re-implementar candle-rendering optimizado, crosshair financiero, time-axis discontinuo (sesiones), etc. No vale el esfuerzo.
- **Custom Canvas/WebGL:** sólo si tu volumen de velas excede 100k visibles simultáneas, lo cual no es tu caso.
- **Plotly:** sí útil para charts de equity curve, distribuciones de drawdown, heatmaps de parámetros — para eso úsalo, no para velas.

5.2 Stack frontend

Next.js 15 (App Router) + React 19 + TypeScript 5.x, sin discusión. Ningún competidor compensa el ecosistema.

State management:

- **TanStack Query v5** para todo lo que es server state (REST, fetch de OHLCV, backtests). Cache, retry, invalidation gratis.
- **Zustand** para client state efímero (UI: símbolo seleccionado, timeframe, layout). API minimalista, no re-renders innecesarios.
- **Valtio o Jotai:** skip. Zustand cubre todo lo que necesitas con menos magia.
- **WebSockets:** cliente directo (`native WebSocket`) O `partysocket` para reconnect/backoff. La conexión a tu propio backend, no a exchanges desde el browser (los exchanges desde el browser exponen tu IP y tienen rate-limits CORS frustrantes).

5.3 Updates en tiempo real eficientes

Patrón obligatorio: delta updates, no full rerenders.

- El backend mantiene la conexión WS al exchange (CCXT Pro), normaliza, y emite a tu frontend **diffs:** `{op: 'kline_update', symbol, tf, candle}` O `{op: 'tick', symbol, price}` .
- En el front, el chart de Lightweight Charts tiene API `series.update(candle)` que muta la última vela in-place sin redibujar el chart entero. Esa es la API que usas; no `series.setData(allCandles)` .
- Throttle a 4–10 updates/s por símbolo en el chart visible (más es invisible al ojo y desperdicia CPU).

- Mantén cada chart en su propio “channel” de Zustand para evitar que un update en BTC re-renderice el card de ETH.

6. Backend e infra

6.1 Lenguajes: Python para todo el dominio (FastAPI), Node sólo si tienes razón concreta

Recomendación dura. Monolito modular en Python, **FastAPI + Uvicorn/Granian**, con WebSockets nativos. No splitees Node + Python en v1: la complejidad operativa no compensa, y FastAPI maneja WS perfectamente vía `starlette`. Splitea sólo si encuentras un cuello de botella concreto (e.g., el broadcaster WS del front se vuelve CPU-bound — entonces y sólo entonces, mete un servicio Node/Bun delante).

Estructura:

```
app/
  api/          # routers FastAPI
  agent/        # Pydantic AI agent + tools
  data/         # exchange adapters, normalizers
  indicators/   # cálculo (polars-ta wrappers, custom)
  research/     # vectorbt strategies, CPCV
  live/         # nautilus integration, paper engine
  alerts/       # rules engine
  journal/      # trade logs, bias detection
  storage/      # repositories sobre Postgres/Timescale/Redis
  llm/          # gateway, prompts, guardrails
```

Microservicios: **no** hasta que un componente claramente se beneficie. La regla es “modular monolith first”. Cuando un componente (típicamente el data ingestor de WS) escale por sí solo, lo extraes.

6.2 Bases de datos

Stack opinado:

- **PostgreSQL 17 + TimescaleDB 2.18 + pgvector 0.9 + pgvector scale**: tu DB primaria. Hypertables para OHLCV/ticks/funding/OI. `OneUptime` Continuous aggregates para resamplings (5m → 1h → 1d). `Tinybird` pgvector para journal embeddings. Una sola DB para 95 % del tráfico es operacionalmente brutal a tu escala.
- **Redis 7+**: cache (con TTL por timeframe), pub/sub para alerts y broadcasting interno,

sessions, rate-limit del LLM gateway.

- **ClickHouse: sólo si** vas a almacenar tick data raw (>1B rows). En v1, no. TimescaleDB con compresión cubre OHLCV multi-timeframe sin sudar.
- **DuckDB / Polars en local** (no DB persistente): para análisis ad-hoc en notebooks. Lee Parquet/Arrow desde S3/local; brutal de rápido.

Patrón de almacenamiento:

- Ticks/trades crudos → opcional, sólo si haces microstructure research.
- OHLCV 1m → hypertable comprimida (TimescaleDB columnar compression, 90–95 % savings).
- Resamplings → continuous aggregates refrescados cada 1 min.
- Funding/OI/Liquidations → hypertables propias.

6.3 Procesamiento async: `asyncio` + `arq` y nada más en v1

- `asyncio` en proceso para todo lo que es real-time (WS handlers, agent loops, FastAPI requests).
- `arq` (Redis-backed, autor de `aiohttp`) para tareas en background: backfills, batch backtests, refresh de caches, generación de embeddings nightly.
- **No metas Celery**: pesado, mal fit con `asyncio`, configuración compleja.
- **No metas Temporal** en v1: brutal pero overkill hasta que tengas workflows multi-día con compensaciones (no es el caso ahora). Excelente cuando crezcas — guárdalo en el roadmap de fase 4.
- **Dramatiq / RQ**: alternativas a `arq`. Si te resultan más cómodos, OK; `arq` encaja mejor con stack 100 % async.

6.4 Sistema de alertas: rules engine event-driven, cero polling

Patrón:

1. **Ingestor único** mantiene WS a exchanges + APIs (Coinglass, etc.) y publica a Redis Streams/PubSub canales `mkt:{exchange}:{symbol}:{type}` (tick, kline, funding, oi, liq).
2. **Rules engine** (servicio in-process) suscrito a esos canales evalúa expresiones declarativas que el usuario crea desde la UI o el chat:

```
- name: "BTC RSI oversold + funding negativo"  
  symbol: BTCUSDT
```

```

when:
  all:
    - indicator.rsi(15m) < 30
    - funding(binance) < -0.02
    - vol_z(1h) > 1.5
cooldown: 30m
channels: [ui, telegram]

```

3. Compila la regla a un AST evaluable (usar `simpleeval` o un mini-DSL custom). Mantén estado mínimo necesario (último valor de cada indicador) en Redis con TTL.
4. Cuando dispara, publica `alert:fired` que el frontend escucha vía WS y, opcionalmente, push a Telegram (`python-telegram-bot`) o Discord.

Nada de cron-jobs cada minuto consultando: eso es polling masivo, latente y consume API limits.

7. Design patterns del copiloto

7.1 Anatomía de una “señal” presentada al trader

Cada output del copiloto cuando habla de oportunidad cumple **este contrato Pydantic**:

```

class TradeIdea(BaseModel):
    symbol: str
    side: Literal['long', 'short']
    timeframe: str
    entry: PriceZone          # rango, no punto
    invalidation: float       # stop lógico, no porcentaje arbitrario
    targets: list[Target]     # con probabilidad y R-multiple
    confidence: float         # 0..1, calibrado
    confluences: list[Confluence] # qué indicadores/factores soportan
    invalidation_reasoning: str
    historical_analog: BacktestSummary | None # similar setup en histórico
    risks: list[str]          # qué la rompería
    sources: list[ToolCitation]
    regime_context: MarketRegime # bull/bear/range, vol, funding env

```

El frontend renderiza esto como una **card** estructurada, no como prosa. Y el LLM nunca llena `confidence` de la nada: viene de un **scorer determinístico** sobre las confluencias (e.g., +0.15 por cada confluencia validada multi-TF, capped en 0.85; nada llega a 0.95+ porque el mercado).

7.2 Multi-timeframe confluence como filtro obligatorio

Regla del producto: **ningún setup se considera "alta calidad" sin alinear ≥ 3 timeframes**. Implementa un tool `get_multi_tf_confluence(symbol)` que retorna el bias por TF (15m, 1h, 4h, 1d) basado en EMAs + estructura. El system prompt del agente exige usarlo antes de proponer un trade.

7.3 Paper trading mode integrado

Todos los trades propuestos por el copiloto pueden materializarse en **paper portfolio** (motor sobre Nautilus en modo simulado, fills usando la WS real del exchange). Cada trade paper se journaliza idéntico a uno real. **Antes de pasar a money real, exiges N trades paper con expectancy positiva en R** (por ejemplo, ≥ 40 trades con expectancy $\geq 0.2R$). Sin eso, el copiloto rechaza activar live.

7.4 Trade journal automático + detección de sesgos

Cada trade cerrado se enriquece automáticamente con metadata: regime de mercado en entry, indicadores en entry/exit, news 24 h antes, tiempo desde último trade, P&L del día/semana, drawdown actual. Esto se almacena estructurado en Postgres y embebido en pgvector.

Detector de sesgos algorítmico (job nocturno + alertas en sesión):

```
def detect_biases(trader_id, window='30d') -> list[BiasFlag]:
    flags = []
    trades = load_trades(trader_id, window)

    # Revenge trading: trade nuevo <X min después de pérdida, tamaño anómalo
    if has_pattern(trades, "loss_then_oversize_within_15min"):
        flags.append(BiasFlag('revenge_trading', severity, examples))

    # Overtrading: n° trades/día > p95 histórico personal
    if today_trade_count > p95(trader_id, 'daily_trades'):
        flags.append(...)

    # FOMO: entradas fuera de zona definida + después de impulso >2*ATR
    ...

    # Sobre-apalancamiento: leverage usado > 2× promedio personal
    ...

    # Cutting winners / running losers (disposition effect)
    if avg_win_R < avg_loss_R and win_rate > 0.55:
        flags.append('disposition_effect', ...)
```

```
return flags
```

El copiloto integra esto en el **inicio de cada sesión**: "Buenas, has hecho 8 trades ayer (tu promedio es 3) y 5 fueron tras pérdidas. ¿Quieres revisar antes de seguir?" Esto, con detector calibrado a tu propia historia, es el feature más diferenciador del producto.

8. Anti-patrones y trampas

Por qué fracasan la mayoría de "AI trading bots":

1. **Tratar al LLM como oráculo de precios.** "Claude, ¿sube BTC mañana?" El LLM no sabe y nunca sabrá; alucinará una respuesta confiada. **Patrón correcto:** el LLM razona sobre datos que tools le proporcionan, jamás genera precios o probabilidades de su cabeza. Implementa esto a nivel system prompt y a nivel guard (validator) que rechaza outputs con números no citados a un tool.
2. **Curve fitting catastrófico.** Encontrar Sharpe 4 en 3 años de BTC sin walk-forward ni CPCV. Lo que vemos en r/algotrading constantemente. Tu pipeline (§4.5) es la inmunización.
3. **Look-ahead bias.** Indicador calculado sobre el **close** y trade ejecutado en ese mismo bar. Solución: shift +1 bar o trade en `open(t+1)`. Auditable en backtest con tests unitarios que rompen si el backtest "ve" datos futuros.
4. **Survivorship bias.** Backtestear sobre el universo *actual* de tokens listados. Solución: dataset point-in-time (cuándo se listó/deslistó cada par). En cripto es más leve que en equities pero existe (Terra, FTT, etc.).
5. **No modelar funding en perps.** Estrategias long-bias que parecen rentables hasta que descuentas 30 % anual de funding. Modélalo desde el día 1.
6. **Race conditions** entre el WS feed y la lógica de decisión. Ej.: indicador sobre la vela en curso (incompleta). Marca explícitamente `is_closed` en cada kline y nunca dispare signals sobre velas no cerradas (excepto si el setup lo requiere y lo declaras).
7. **Exchange disconnects no manejados.** Pierdes 30 segundos de WS, tu cache de orderbook está obsoleto, una alerta dispara con datos rancios. Implementa heartbeat + watchdog que invalida cache y forza resnap si `gap > N` segundos.
8. **Partial fills** ignorados en simulación. Una limit grande en un activo ilíquido se llena 30 %; tu estrategia asumió 100 %. Nautilus modela esto bien si lo configuras.

9. **No diferenciar señal de ejecución.** El copiloto te puede dar la señal correcta y aun así perdías por una entrada mal manejada. Los KPIs separados (signal expectancy vs realized expectancy) detectan la diferencia.
 10. **Tool sprawl.** Añadir 80 tools al agente porque “podrían ser útiles”. El LLM se confunde, latencia sube, costes explotan. **Mantén ≤ 25 tools en el agente activo**, agrupadas por dominio.
-

9. Ecosistema 2026 (qué adoptar y qué ignorar)

Adoptar:

- **NautilusTrader** (Rust core, Apache 2.0): [Pytrade](#) el motor de ejecución serio open-source de la era 2024-2026.
- **VectorBT PRO**: research powerhouse.
- **mlfinlab** (Hudson & Thames): implementaciones canónicas de López de Prado (CPCV, fractional differentiation, triple-barrier labeling). Indispensable.
- **CCXT Pro**: ya cubierto.
- **Pydantic AI**: ya cubierto.
- **Hummingbot**: si más adelante quieres meterte en market-making, es la base.
[CoinCodeCap](#) Para copiloto direccional, no relevante v1.
- **Freqtrade**: extensa comunidad, [GitHub](#) FreqAI integra ML. Útil para inspirarte en cómo estructurar configs y pairlists, pero **no lo uses como base** del copiloto — su modelo de programación (strategies como clases con buy/sell hooks) es restrictivo para un producto conversacional.

Mantener distancia (en v1):

- **Jesse**: bonito, pero ecosistema más pequeño que NautilusTrader y peor performance.
- **OctoBot**: orientado a usuario final no developer.

Frameworks LLM-for-finance:

- **BloombergGPT**: cerrado, irrelevante para ti.
- **FinGPT**: open-source, interesante académicamente. En benchmarks 2025-2026 los modelos frontier (GPT-5.5, Opus 4.7, Gemini 3.1) **superan a FinGPT** incluso en tareas financieras, porque entrenaron sobre corpus mucho mayores que incluyen toda la web

financiera pública. [Awesome Agents](#) **No fine-tunees un FinLLM en v1.** Usa frontier + RAG bien hecho + tools.

- **FinRobot:** agente multi-tool específico financiero. Inspírate, no lo uses como base — tu Pydantic AI + tools custom es más limpio y específico a cripto.
- **FinanceBench:** úsalo para evaluar tu agente en QA financiero (es un eval set, no un framework).

MCP (Model Context Protocol): dado que tu cerebro principal es Claude, **expón tus tools como un MCP server** además de como tools del agente Pydantic AI. Beneficio: puedes consumir tu copiloto desde Claude Desktop, Cursor, o cualquier cliente MCP, sin reescribir nada. Es trivial con el SDK de Python de Anthropic. Esto es lo que hace TradeStation con MCP [TradeStation](#) en febrero 2026 y la dirección clara del ecosistema.

10. Roadmap incremental — Definition of Done por fase

El error reportado ("empiezas a añadir indicadores y te pierdes") es real y se resuelve **bloqueando el alcance por fase y exigiendo DoD verificable antes de avanzar.**

Fase 0 — Cimientos (semana 1-2)

Objetivo: infraestructura mínima funcional, sin chat.

- Repositorio mono-repo (Turborepo): `/api` (FastAPI + Pydantic AI), `/web` (Next.js 15), `/packages/shared-types` (TS types generados desde schemas Pydantic).
- Postgres + TimescaleDB + Redis con docker-compose.
- Adapter Binance (Spot + USDT-M Futures) con CCXT Pro: `fetch_ohlcv` + `watch_ohlcv` funcionando.
- Hypertable `ohlcv(symbol, timeframe, ts, o, h, l, c, v)` con compresión y retention policy.
- Backfill de BTCUSDT 1m de 2 años en local.
- Frontend con un solo chart (Lightweight Charts) renderizando BTCUSDT 1h.

DoD: desde el browser ves BTCUSDT 1h, las velas se actualizan en vivo, y existen 2 años de históricos en DB.

Fase 1 — Análisis on-demand (semanas 3-4)

Objetivo: "dame análisis de ETH 4h" funciona bien.

- Pydantic AI agent con Sonnet 4.6 y los siguientes tools: `get_ohlcv`, `get_indicators`, `get_market_structure`, `get_multi_tf_confluence`, `get_funding_rate`, `get_open_interest`.
- 8 indicadores core implementados con polars-ta + tests vs TA-Lib (correlación >0.999).
- Endpoint `POST /chat` que recibe mensaje + contexto, devuelve stream SSE con tool calls visibles.
- Frontend con chat UI básico (Vercel AI SDK chat hooks o equivalente custom) y rendering de `TradeIdea` cards.

DoD: preguntas “analiza ETH en 4h” y recibes una `TradeIdea` con confluencias, indicadores citados, niveles concretos. Métrica de éxito: el output siempre cita ≥ 1 tool por afirmación cuantitativa (validador automático).

Fase 2 — Backtesting y journal (semanas 5-7)

Objetivo: investigar estrategias y journalizar trades manuales.

- Integración VectorBT PRO. Una estrategia base “EMA cross + ATR stop” implementada.
- Pipeline `backtest` → `walk_forward` → `CPCV` → `metrics` con deflated Sharpe + PSR.
- Tabla `trades` y endpoint para registrar trades manuales (vía CSV import o entrada UI).
- pgvector + voyage-3-large para embeddings de trades.
- Tool `get_similar_past_trades(setup_features)`.
- UI: vista “Research” (params sweep, equity curves) + vista “Journal” con filtros.

DoD: corres un backtest desde la UI, obtienes Sharpe + Deflated Sharpe + Calmar + max DD + curva. Al cerrar un trade manual, el sistema lo embebe y al preguntar “¿he hecho un setup similar antes?” devuelve top-3 con outcomes.

Fase 3 — Alertas y monitoreo activo (semanas 8-9)

Objetivo: no estar pegado a la pantalla.

- Rules engine con DSL declarativo, evaluación event-driven.
- UI para crear alertas conversacionalmente (“avísame si BTC pierde 60k con vol > promedio”).
- El copiloto puede crear/listar/eliminar alertas vía tools.
- Push a Telegram.

- Watchdog de WS (heartbeat, reconnect, cache invalidation).

DoD: creas una alerta por chat, la regla se persiste, dispara en condiciones de prueba (con replay histórico) y llega a Telegram.

Fase 4 — Paper trading + detector de sesgos (semanas 10-12)

Objetivo: copiloto operativo end-to-end.

- Motor paper trading sobre Nautilus en modo simulado con WS real.
- "Activar setup" desde una `TradeIdea` crea posición paper con stop/targets reales.
- Trade journal automático de paper trades.
- Job nocturno + tool `detect_bias_patterns`. Sesgos cubiertos en v1: overtrading, revenge trading, FOMO entries (fuera de zona), oversize ($>2\times$ avg), disposition effect.
- Briefing matutino del copiloto: "ayer 6 trades, 4 paper, 1 revenge flag, drawdown 2 %".

DoD: abres y cierras 10 trades paper en una semana, el journal está completo, el detector de sesgos identifica al menos 1 patrón real y el briefing matutino se genera automáticamente.

Fase 5 — On-chain, derivados avanzados, RAG playbook (semanas 13-15)

- Integración Glassnode + Coinglass (funding histórico, OI heatmaps, liquidations).
- Tools on-chain: `get_onchain_metric`, `get_whale_movements`.
- RAG sobre playbook de estrategias propias.
- Multi-modelo router (Sonnet default, escalada a Opus para deep dive).

DoD: "¿qué setups históricos similares al actual de BTC con MVRV alto + funding caliente acabaron en corrección?" devuelve análisis con datos cited.

Fase 6 — Producción personal estable (semanas 16+)

- Observabilidad: Langfuse o Helicone para trazas de LLM, Grafana sobre Postgres/Timescale para infra, Sentry para errores.
- Cost guards: límites de gasto/día por modelo.
- Backups automáticos de Postgres a S3.
- (Opcional) Exposición de tools como MCP server para usar desde Claude Desktop.
- (Opcional, MUY a futuro) Live trading con confirmation handshake estricto.

DoD: sistema corre 30 días sin intervención manual, logs sanos, costes bajo control, copiloto te ha ahorrado al menos un trade malo (subjetivo pero el norte del producto).

Apéndice: presupuesto estimado mensual (uso personal serio)

Concepto	Coste/mes
Anthropic API (Sonnet 4.6 chat + Opus 4.7 deep dive)	\$40-100
Voyage AI embeddings + reranker	\$10-20
CCXT Pro license (anualizado)	~\$25
VectorBT PRO (anualizado)	~\$45
Coinglass API básico	\$35
Glassnode Advanced	\$39
VPS (Hetzner CCX23 o similar, 4 vCPU/16 GB)	\$30-50
Postgres managed (opcional, Neon/Supabase)	\$0-25
Total	~\$225-340/mes

Para un trader que mueve cuentas serias, es ruido. Para uno hobby, paper-trade primero y baja a Sonnet-only + Glassnode free + Coinglass free, y bajas a ~\$70.

Cierre opinado

Lo que separa este blueprint de los 50 tutoriales de YouTube “construye tu trading bot con ChatGPT”:

1. El LLM es **interpretador y orquestador**, jamás oráculo. Toda afirmación cuantitativa cita un tool determinístico.
2. **Doble stack de backtesting** (vectorial para investigar, event-driven para validar) y **CPCV + Deflated Sharpe** obligatorios — no “Sharpe 4 en backtest, fly to the moon”.
3. **Datos de derivados y on-chain como ciudadanos de primera**, no afterthought. En cripto, esto es la mitad del edge.

4. **Journal automatizado + detector de sesgos personal:** el feature que más mejora a un trader real, y nadie lo construye bien porque exige integración profunda.
5. **Paper-mandatorio** antes de live, con DoD numérico.
6. **Frontend opinado** sobre Lightweight Charts + Next.js + delta updates.

Si lo construyes en este orden y respetas los DoD, en 16 semanas tienes un copiloto que objetivamente te hace mejor trader. Si te saltas el orden, en 16 semanas tienes un demo bonito y cero alpha.

Empieza por Fase 0 el lunes.